Manual Técnico - Proyecto 1 SO1

Desarrollo, Conexión y Gestión de Contenedores en Entornos Virtualizados

Carnet: 202110378
Fecha: Agosto 2026
Curso: Sistemas Operativos 1
Universidad: Universidad San Carlos de Guatemala - Facultad de Ingeniería

Tabla de Contenido

1. Introducción
2. Arquitectura del Sistema
3. Configuración de Máquinas Virtuales
4. Configuración de Runtimes
5. Desarrollo de las APIs
6. Configuración del Registro Zot
7. Comunicación entre APIs
8. Pruebas y Validación
9. Guía de Instalación
10. Solución de Problemas

1. Introducción

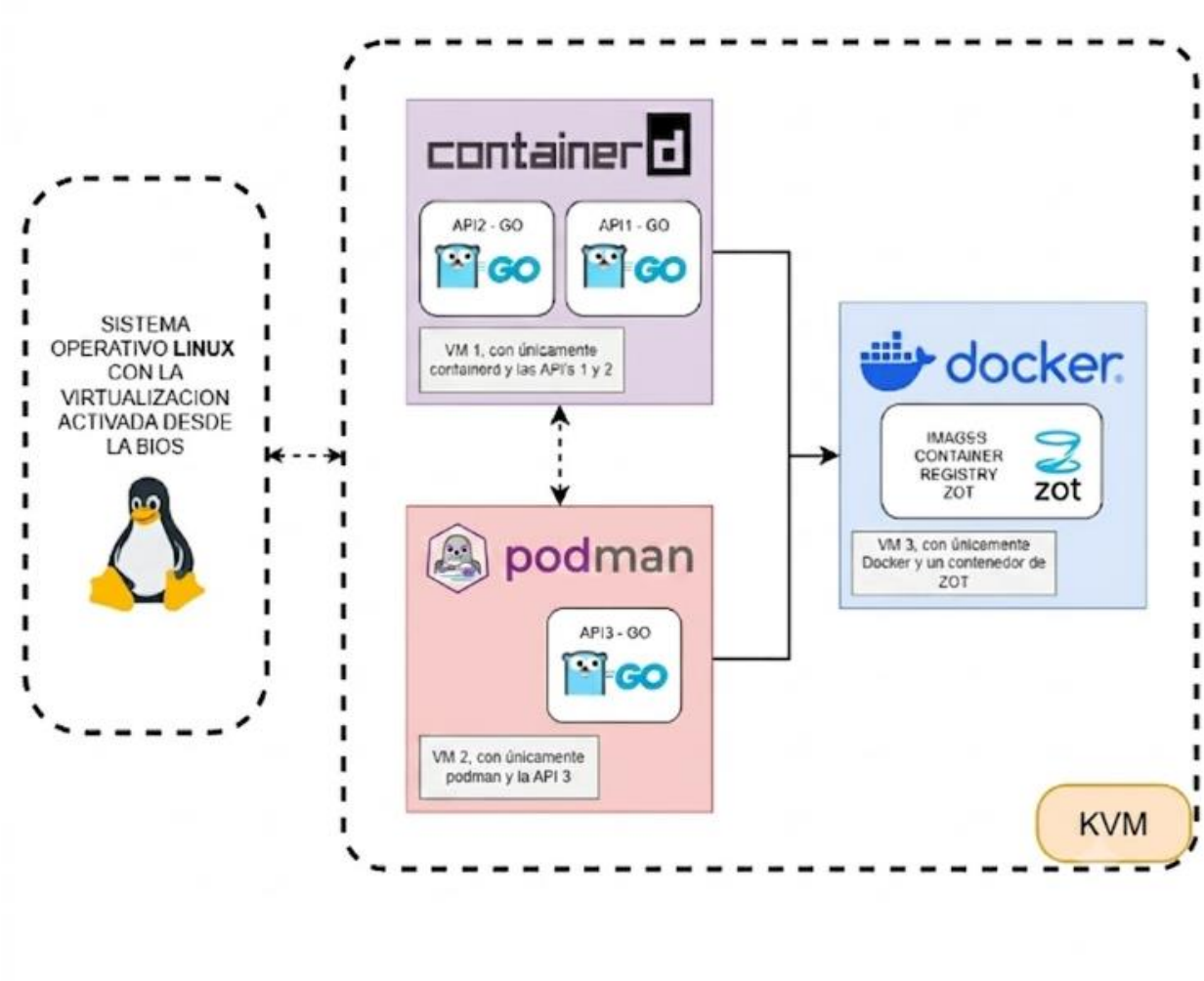
Este proyecto implementa un entorno virtualizado con 3 máquinas virtuales (VMs) utilizando KVM, cada una ejecutando contenedores con diferentes runtimes:

- **VM1:** Containerd (Nerdctl) - API1
- **VM2:** Podman - API2
- **VM3:** Docker + Zot Registry - API3

El sistema implementa una comunicación REST/HTTP entre 3 APIs desarrolladas en Go, con un registro privado de imágenes (Zot) para la distribución de contenedores.

2. Arquitectura del Sistema

Diagrama de Arquitectura



3. Configuración de Máquinas Virtuales

Especificaciones de las VMs

VM	IP	Runtime	API	Puerto
VM1	192.168.122.254	Containerd/Nerdctl	API1	5001
VM2	192.168.122.101	Podman	API2	5002
VM3	192.168.122.245	Docker	API3	5003

Recursos asignados

VM	RAM	CPUs	Disco	SO
VM1	512MB	2	5GB	Alpine Linux
VM2	512MB	2	5GB	Alpine Linux
VM3	512MB	2	5GB	Alpine Linux

Configuración de Red

Todas las VMs están configuradas en modo **bridge** para permitir comunicación entre ellas:

bash

```
# Verificar conectividad
ping -c 3 192.168.122.254
ping -c 3 192.168.122.101
ping -c 3 192.168.122.245
```

4. Configuración de Runtimes

VM1 - Containerd/Nerdctl

bash

```
# Instalación
apk add containerd nerdctl buildkit

# Iniciar servicios
rc-service containerd start
rc-service buildkitd start
rc-update add containerd default
rc-update add buildkitd default

# Verificación
nerdctl version
nerdctl ps
```

Configuración de BuildKit:

bash

```
# Crear configuración para BuildKit
mkdir -p /etc/buildkit
cat > /etc/buildkit/buildkitd.toml << EOF
[worker.oci]
  env = ['GODEBUG=netdns=go']
  dns = ['8.8.8.8', '1.1.1.1']
EOF

rc-service buildkitd restart
```

VM2 - Podman

bash

```
# Instalación
apk add podman buildah

# Configuración de cgroups
cat > /etc/containers/containers.conf << EOF
[containers]
cgroup_manager = "cgroupfs"
EOF

export PODMAN_IGNORE_CGROUPSV1_WARNING=1

# Verificación
podman --version
podman ps
```

VM3 - Docker

bash

```
# Instalación
apk add docker docker-cli-compose

# Iniciar servicio
rc-service docker start
rc-update add docker boot

# Verificación
docker --version
docker ps
```

5. Desarrollo de las APIs

5.1 Estructura de las APIs

Todas las APIs siguen la misma estructura:

text

```
/api#-202110378/
├──── main.go          # Código fuente en Go
├──── Dockerfile       # Dockerfile para construcción
└──── README.md        # Documentación de la API
```

5.2 Endpoints

API	Health	Llamada 1	Llamada 2
API1	/health	/api1/202110378/call-api2	/api1/202110378/call-api3

API	Health	Llamada 1	Llamada 2
API2	/health	/api2/202110378/call-api1	/api2/202110378/call-api3
API3	/health	/api3/202110378/call-api1	/api3/202110378/call-api2

5.3 Código Fuente

VM1 - API1 (/api1-202110378/main.go)

go

```
package main

import (
        "encoding/json"
        "fmt"
        "net/http"
        "time"
)

const (
        carnet = "202110378"
        vm     = "VM1"
        port   = ":5001"
)

// IPs de las otras VMs
const (
        vm2IP = "192.168.122.101"
        vm3IP = "192.168.122.245"
)

type HealthResponse struct {
        Status    string `json:"status"`
        Message   string `json:"message"`
        Timestamp string `json:"timestamp"`
        VM        string `json:"VM"`
        Carnet    string `json:"carnet"`
}

type CallResponse struct {
        Apiname    string `json:"apiname"`
        Message    string `json:"message"`
        Connection bool   `json:"connection"`
        Carnet     string `json:"carnet"`
}

func healthHandler(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(HealthResponse{
                Status:    "UP",
                Message:   "API1 is Ready",
                Timestamp: time.Now().Format(time.RFC3339),
```

```
                VM:        vm,
                Carnet:    carnet,
        })
}

func callAPI2Handler(w http.ResponseWriter, r *http.Request) {
        httpClient := &http.Client{Timeout: 3 * time.Second}
        resp, err := httpClient.Get(fmt.Sprintf("http://%s:5002/health", vm2IP))
        if err == nil && resp.StatusCode == 200 {
                var health HealthResponse
                json.NewDecoder(resp.Body).Decode(&health)
                resp.Body.Close()
                if health.Status == "UP" {
                        json.NewEncoder(w).Encode(CallResponse{
                                Apiname:    "API2",
                                Message:    "The API2 located on the VM2 is working",
                                Connection: true,
                                Carnet:     carnet,
                        })
                        return
                }
        }
        json.NewEncoder(w).Encode(CallResponse{
                Apiname:    "API2",
                Message:    "ERROR: The API2 located on the VM2 is not working",
                Connection: false,
                Carnet:     carnet,
        })
}

func callAPI3Handler(w http.ResponseWriter, r *http.Request) {
        httpClient := &http.Client{Timeout: 3 * time.Second}
        resp, err := httpClient.Get(fmt.Sprintf("http://%s:5003/health", vm3IP))
        if err == nil && resp.StatusCode == 200 {
                var health HealthResponse
                json.NewDecoder(resp.Body).Decode(&health)
                resp.Body.Close()
                if health.Status == "UP" {
                        json.NewEncoder(w).Encode(CallResponse{
                                Apiname:    "API3",
                                Message:    "The API3 located on the VM3 is working",
                                Connection: true,
                                Carnet:     carnet,
                        })
                        return
                }
        }
        json.NewEncoder(w).Encode(CallResponse{
                Apiname:    "API3",
                Message:    "ERROR: The API3 located on the VM3 is not working",
                Connection: false,
                Carnet:     carnet,
```

```
        })
}

func main() {
        http.HandleFunc("/health", healthHandler)
        http.HandleFunc("/api1/202110378/call-api2", callAPI2Handler)
        http.HandleFunc("/api1/202110378/call-api3", callAPI3Handler)

        fmt.Println("API1 corriendo en puerto 5001")
        http.ListenAndServe(port, nil)
}
```

VM1 - Dockerfile

dockerfile

```
FROM alpine:latest
WORKDIR /app
COPY api1 .
EXPOSE 5001
CMD ["./api1"]
```

VM2 - API2 (/api2-202110378/main.go)

go

```
package main

import (
        "encoding/json"
        "fmt"
        "net/http"
        "time"
)

const (
        carnet = "202110378"
        vm     = "VM2"
        port   = ":5002"
)

// IPs de las otras VMs
const (
        vm1IP = "192.168.122.254"
        vm3IP = "192.168.122.245"
)

type HealthResponse struct {
        Status    string `json:"status"`
        Message   string `json:"message"`
        Timestamp string `json:"timestamp"`
        VM        string `json:"VM"`
        Carnet    string `json:"carnet"`
```

```
}

type CallResponse struct {
        Apiname    string `json:"apiname"`
        Message    string `json:"message"`
        Connection bool   `json:"connection"`
        Carnet     string `json:"carnet"`
}

func healthHandler(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(HealthResponse{
                Status:    "UP",
                Message:   "API2 is Ready",
                Timestamp: time.Now().Format(time.RFC3339),
                VM:        vm,
                Carnet:    carnet,
        })
}

func callAPI1Handler(w http.ResponseWriter, r *http.Request) {
        httpClient := &http.Client{Timeout: 3 * time.Second}
        resp, err := httpClient.Get(fmt.Sprintf("http://%s:5001/health", vm1IP))
        if err == nil && resp.StatusCode == 200 {
                var health HealthResponse
                json.NewDecoder(resp.Body).Decode(&health)
                resp.Body.Close()
                if health.Status == "UP" {
                        json.NewEncoder(w).Encode(CallResponse{
                                Apiname:    "API1",
                                Message:    "The API1 located on the VM1 is working",
                                Connection: true,
                                Carnet:     carnet,
                        })
                        return
                }
        }
        json.NewEncoder(w).Encode(CallResponse{
                Apiname:    "API1",
                Message:    "ERROR: The API1 located on the VM1 is not working",
                Connection: false,
                Carnet:     carnet,
        })
}

func callAPI3Handler(w http.ResponseWriter, r *http.Request) {
        httpClient := &http.Client{Timeout: 3 * time.Second}
        resp, err := httpClient.Get(fmt.Sprintf("http://%s:5003/health", vm3IP))
        if err == nil && resp.StatusCode == 200 {
                var health HealthResponse
                json.NewDecoder(resp.Body).Decode(&health)
                resp.Body.Close()
```

```
                if health.Status == "UP" {
                        json.NewEncoder(w).Encode(CallResponse{
                                Apiname:    "API3",
                                Message:    "The API3 located on the VM3 is working",
                                Connection: true,
                                Carnet:     carnet,
                        })
                        return
                }
        }
        json.NewEncoder(w).Encode(CallResponse{
                Apiname:    "API3",
                Message:    "ERROR: The API3 located on the VM3 is not working",
                Connection: false,
                Carnet:     carnet,
        })
}

func main() {
        http.HandleFunc("/health", healthHandler)
        http.HandleFunc("/api2/202110378/call-api1", callAPI1Handler)
        http.HandleFunc("/api2/202110378/call-api3", callAPI3Handler)

        fmt.Println("API2 corriendo en puerto 5002")
        http.ListenAndServe(port, nil)
}
```

VM2 - Containerfile

dockerfile

```
FROM alpine:latest
WORKDIR /app
COPY api2 .
EXPOSE 5002
CMD ["./api2"]
```

VM3 - API3 (/api3-202110378/main.go)

go

```
package main

import (
        "encoding/json"
        "fmt"
        "net/http"
        "time"
)

const (
        carnet = "202110378"
        vm     = "VM3"
```

```
        port   = ":5003"
)

// IPs de las otras VMs
const (
        vm1IP = "192.168.122.254"
        vm2IP = "192.168.122.101"
)

type HealthResponse struct {
        Status    string `json:"status"`
        Message   string `json:"message"`
        Timestamp string `json:"timestamp"`
        VM        string `json:"VM"`
        Carnet    string `json:"carnet"`
}

type CallResponse struct {
        Apiname    string `json:"apiname"`
        Message    string `json:"message"`
        Connection bool   `json:"connection"`
        Carnet     string `json:"carnet"`
}

func healthHandler(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(HealthResponse{
                Status:    "UP",
                Message:   "API3 is Ready",
                Timestamp: time.Now().Format(time.RFC3339),
                VM:        vm,
                Carnet:    carnet,
        })
}

func callAPI1Handler(w http.ResponseWriter, r *http.Request) {
        httpClient := &http.Client{Timeout: 3 * time.Second}
        resp, err := httpClient.Get(fmt.Sprintf("http://%s:5001/health", vm1IP))
        if err == nil && resp.StatusCode == 200 {
                var health HealthResponse
                json.NewDecoder(resp.Body).Decode(&health)
                resp.Body.Close()
                if health.Status == "UP" {
                        json.NewEncoder(w).Encode(CallResponse{
                                Apiname:    "API1",
                                Message:    "The API1 located on the VM1 is working",
                                Connection: true,
                                Carnet:     carnet,
                        })
                        return
                }
        }
```

```
        json.NewEncoder(w).Encode(CallResponse{
                Apiname:    "API1",
                Message:    "ERROR: The API1 located on the VM1 is not working",
                Connection: false,
                Carnet:     carnet,
        })
}

func callAPI2Handler(w http.ResponseWriter, r *http.Request) {
        httpClient := &http.Client{Timeout: 3 * time.Second}
        resp, err := httpClient.Get(fmt.Sprintf("http://%s:5002/health", vm2IP))
        if err == nil && resp.StatusCode == 200 {
                var health HealthResponse
                json.NewDecoder(resp.Body).Decode(&health)
                resp.Body.Close()
                if health.Status == "UP" {
                        json.NewEncoder(w).Encode(CallResponse{
                                Apiname:    "API2",
                                Message:    "The API2 located on the VM2 is working",
                                Connection: true,
                                Carnet:     carnet,
                        })
                        return
                }
        }
        json.NewEncoder(w).Encode(CallResponse{
                Apiname:    "API2",
                Message:    "ERROR: The API2 located on the VM2 is not working",
                Connection: false,
                Carnet:     carnet,
        })
}

func main() {
        http.HandleFunc("/health", healthHandler)
        http.HandleFunc("/api3/202110378/call-api1", callAPI1Handler)
        http.HandleFunc("/api3/202110378/call-api2", callAPI2Handler)

        fmt.Println("API3 corriendo en puerto 5003")
        http.ListenAndServe(port, nil)
}
```

VM3 - Dockerfile

dockerfile

```
FROM alpine:latest
WORKDIR /app
COPY api3 .
EXPOSE 5003
CMD ["./api3"]
```

6. Configuración del Registro Zot

6.1 Instalación y Configuración

bash

```
# Crear directorio para datos
mkdir -p ~/zot/data

# Ejecutar Zot
docker run -d ¥
  -p 5000:5000 ¥
  --name zot-registry ¥
  -v ~/zot/data:/var/lib/registry ¥
  ghcr.io/project-zot/zot:latest

# Verificar
curl http://localhost:5000/v2/
```

6.2 Configuración como Registro Inseguro

VM3 - Docker:

bash

```
cat > /etc/docker/daemon.json << EOF
{
  "insecure-registries": ["192.168.122.245:5000"]
}
EOF

rc-service docker restart
```

VM2 - Podman:

bash

```
cat >> /etc/containers/registries.conf << EOF

[[registry]]
location = "192.168.122.245:5000"
insecure = true
EOF
```

VM1 - Containerd:

bash

```
mkdir -p /etc/containerd/certs.d/192.168.122.245:5000

cat > /etc/containerd/certs.d/192.168.122.245:5000/hosts.toml << EOF
server = "http://192.168.122.245:5000"

[host."http://192.168.122.245:5000"]
```

```
  capabilities = ["pull", "resolve", "push"]
  skip_verify = true
EOF
```

7. Comunicación entre APIs

7.1 Formato de Respuesta

/health:

json

```
{
  "status": "UP",
  "message": "API1 is Ready",
  "timestamp": "2026-08-20T...Z",
  "VM": "VM1",
  "carnet": "202110378"
}
```

Llamada exitosa:

json

```
{
  "apiname": "API2",
  "message": "The API2 located on the VM2 is working",
  "connection": true,
  "carnet": "202110378"
}
```

Llamada fallida:

json

```
{
  "apiname": "API2",
  "message": "ERROR: The API2 located on the VM2 is not working",
  "connection": false,
  "carnet": "202110378"
}
```

7.2 Flujo de Comunicación

1. El usuario realiza una petición a una API (ej: API2)
2. La API recibe la solicitud y prepara una llamada HTTP
3. Intenta contactar el endpoint `/health` de la API destino
4. Analiza la respuesta:

- ⑩ Si `status = "UP"` → respuesta exitosa
- ⑩ Si error o `status != "UP"` → respuesta con error

5. Retorna la respuesta al usuario

8. Pruebas y Validación

8.1 Pruebas de Health

bash

```
# VM1
curl http://192.168.122.254:5001/health

# VM2
curl http://192.168.122.101:5002/health

# VM3
curl http://192.168.122.245:5003/health
```

8.2 Pruebas de Comunicación

bash

```
# API1 → API2
curl http://192.168.122.254:5001/api1/202110378/call-api2

# API1 → API3
curl http://192.168.122.254:5001/api1/202110378/call-api3

# API2 → API1
curl http://192.168.122.101:5002/api2/202110378/call-api1

# API2 → API3
curl http://192.168.122.101:5002/api2/202110378/call-api3

# API3 → API1
curl http://192.168.122.245:5003/api3/202110378/call-api1

# API3 → API2
curl http://192.168.122.245:5003/api3/202110378/call-api2
```

8.3 Pruebas de Zot

bash

```
# Ver repositorios
curl http://192.168.122.245:5000/v2/_catalog

# Push de imagen
docker tag alpine:latest 192.168.122.245:5000/alpine:test
```

```
docker push 192.168.122.245:5000/alpine:test

# Pull de imagen
docker pull 192.168.122.245:5000/alpine:test
```

9. Guía de Instalación

9.1 Prerrequisitos

- ❿ KVM instalado
- ❿ Alpine Linux en las VMs
- ❿ Conexión de red entre VMs

9.2 Instalación en VM1

bash

```
# Clonar repositorio
git clone https://github.com/tu-usuario/202110378_LAB_SO1_2S2026.git
cd 202110378_LAB_SO1_2S2026/VM1-api1-202110378

# Compilar
go mod init api1
go mod tidy
go build -o api1 main.go

# Construir imagen
nerdctl build -t api1-202110378 .

# Ejecutar
nerdctl run -d --name api1-202110378 -p 5001:5001 api1-202110378
```

9.3 Instalación en VM2

bash

```
# Clonar repositorio
git clone https://github.com/tu-usuario/202110378_LAB_SO1_2S2026.git
cd 202110378_LAB_SO1_2S2026/VM2-api2-202110378

# Compilar
go mod init api2
go mod tidy
go build -o api2 main.go

# Construir imagen
podman build -f Containerfile -t api2-202110378 .

# Ejecutar
podman run -d --name api2-202110378 -p 5002:5002 api2-202110378
```

9.4 Instalación en VM3

bash

```
# Clonar repositorio
git clone https://github.com/tu-usuario/202110378_LAB_SO1_2S2026.git
cd 202110378_LAB_SO1_2S2026/VM3-api3-202110378

# Compilar
go mod init api3
go mod tidy
go build -o api3 main.go

# Construir imagen
docker build -t api3-202110378 .

# Ejecutar
docker run -d --name api3-202110378 -p 5003:5003 api3-202110378
```

10. Solución de Problemas

10.1 Error de conexión entre VMs

bash

```
# Verificar conectividad
ping -c 3 192.168.122.254
ping -c 3 192.168.122.101
ping -c 3 192.168.122.245

# Verificar puertos abiertos
netstat -tulpn | grep 5001
netstat -tulpn | grep 5002
netstat -tulpn | grep 5003
```

10.2 Error de DNS en BuildKit

bash

```
# Configurar DNS
cat > /etc/buildkit/buildkitd.toml << EOF
[worker.oci]
  dns = ['8.8.8.8', '1.1.1.1']
EOF

rc-service buildkitd restart
```

10.3 Error de cgroups en Podman

bash

```
# Configurar cgroups
```

```
export PODMAN_IGNORE_CGROUPSV1_WARNING=1
podman build --cgroup-manager=cgroupfs -t api2-202110378 .
```

Anexos

Anexo A: Comandos Rápidos

bash

```
# Reiniciar todas las APIs
# VM1
nerdctl restart api1-202110378
# VM2
podman restart api2-202110378
# VM3
docker restart api3-202110378

# Ver logs
# VM1
nerdctl logs api1-202110378
# VM2
podman logs api2-202110378
# VM3
docker logs api3-202110378
```

Anexo B: Variables de Entorno

Variable	Valor
CARNET	202110378
VM1_IP	192.168.122.254
VM2_IP	192.168.122.101
VM3_IP	192.168.122.245
ZOT_PORT	5000